Phase 2 — Grade 5: Regression

Detailed Revision Guide — full formula breakdowns, edge cases, and common-mistake debugging

1. Multiple Linear Regression

Extends single-feature regression to MULTIPLE features at once: y-hat = m1x1 + m2x2 + ... + mnxn + b.

The dot-product structure (callback to Linear Algebra, Grade 1)

`y-hat = m·x + b where m=[m1,m2,...] (weight vector), x=[x1,x2,...] (feature vector)`

`This is EXACTLY the dot product from Grade 1, plus a bias term -- and exactly a neuron's computation (dot(inputs,weights)+bias) previewed back then.`

Worked Example: house price from size & bedrooms, m1=30,m2=5,b=10

`size=1.5, bedrooms=3: y-hat = 30(1.5)+5(3)+10 = 45+15+10 = 70`

`size=2, bedrooms=4: y-hat = 30(2)+5(4)+10 = 60+20+10 = 90`

TIP: Everything from Grade 4 still applies unchanged
MSE and gradient descent work identically -- just with ONE partial derivative per weight now (∂MSE/∂m1, ∂MSE/∂m2, ..., ∂MSE/∂b) instead of a single m. The mechanics don't change, only the number of parameters being updated each step.

2. R-Squared (R2) — Interpretable Model Evaluation

MSE alone is hard to judge -- is MSE=50 good or bad? It depends entirely on the data's scale. R2 fixes this by expressing fit as a PERCENTAGE: what fraction of the data's variation does the model explain?

Formula

`R2 = 1 - (SSE / SST)`

`SSE = the model's own sum of squared errors (same as Grade 4)`

`SST = ∑(Actual - mean of all Actual)^2 -- how much the data varies if you just guessed the mean every time, with no model at all`

Worked Example: Grade 4 data (actual: 22,29,41,48), model SSE=6

`Step 1 -- mean of Actual = (22+29+41+48)/4 = 140/4 = 35`

`Step 2 -- SST = (22-35)^2+(29-35)^2+(41-35)^2+(48-35)^2`

`= 169+36+36+169 = 410`

`Step 3 -- R2 = 1 - 6/410 = 1 - 0.0146 = 0.9854 (98.5% of variation explained -- excellent fit)`

A worse model with SSE=200 on the SAME data: R2 = 1 - 200/410 = 0.5122 (only 51% explained)

TIP: The intuition behind SST as a 'baseline of ignorance'
SST represents the error you'd get by using the simplest possible 'model' -- always predicting the mean, regardless of input. If your real model's SSE is much smaller than this baseline SST, R2 approaches 1 (you're doing much better than guessing blindly). If SSE ≈ SST, R2 approaches 0 (your model isn't adding any real value over just guessing the average).

EDGE CASE: R2 CAN be negative (not covered in hands-on, but a real possibility worth knowing)
If a model's SSE is actually WORSE than SST (meaning the model's predictions are even more off than simply guessing the mean every time -- which can happen with a badly misspecified or badly-trained model), then SSE/SST > 1, making R2 = 1 - (something >1) a NEGATIVE number. A negative R2 is a strong signal the model is actively worse than doing nothing.

EDGE CASE: R2 = 1 requires SSE = 0 exactly (every single prediction perfect)
Same strict requirement as Grade 4's MSE=0 edge case -- since SSE is a sum of squared (always non-negative) terms, R2=1 only happens when every individual error is exactly zero.

3. Polynomial Regression

When data genuinely CURVES, a straight line has a high MSE no matter how it's adjusted -- the SHAPE itself is wrong, not just the specific m,b values. The fix: add higher powers of x as additional features.

Formula

y-hat = m1*x + m2*x^2 + b

This is STILL ordinary linear regression -- x^2 is simply treated as another input feature (exactly like 'bedrooms' was an additional feature in multiple regression). Same MSE, same gradient descent, completely unchanged mechanics.

COMMON MISTAKE: Assuming polynomial regression needs a fundamentally different training method
It's natural to assume curving the fit requires new math, since the OUTPUT shape looks different. But the training PROCESS (compute MSE, compute gradients, step downhill) is byte-for-byte identical -- only the INPUT features change (you feed in x and x^2 as two separate columns, instead of just x). Nothing about gradient descent itself needs to change.

Why does adding x^2 let the model curve? (the mechanism)

x^2 grows FASTER than x as x increases (and behaves differently for negative x than positive x, unlike x itself). Giving the model a weight (m2) to attach to this faster-growing, differently-shaped term lets its output bend upward or downward instead of being forced into one constant rate of change (a straight line's defining limitation).

COMMON MISTAKE: Believing more training time can fix a straight line fit to curved data
Real example from practice reasoning (correctly avoided): y-hat=mx+b has exactly two adjustable numbers, m and b -- neither can introduce curvature, no matter how many gradient descent steps run. Training only finds the BEST straight line; if the true relationship curves, even the best possible straight line has an irreducible error floor that no amount of additional training can lower. Only adding a genuinely non-linear feature (x^2, x^3, etc.) can fix a shape mismatch.

EDGE CASE: Higher-degree polynomials can overfit (flagged, not yet covered in depth)
Adding MANY powers of x (x, x^2, x^3, x^4, ...) lets the model fit the TRAINING data increasingly closely -- potentially memorizing noise rather than learning the true pattern (overfitting, first flagged in Grade 4). A model that fits training data perfectly via a high-degree polynomial may perform WORSE on the held-out test set than a simpler one. Choosing the right polynomial degree is a real trade-off addressed more fully in later grades (e.g. cross-validation, regularization).

4. Full Debugging Checklist for This Grade

---

1. For multiple regression, have I matched each weight to its correct corresponding feature (not mixed up m1 with x2, etc.)? 2. For R2, have I computed SST using the MEAN of actual values, not some other reference point? 3. Have I kept SSE and SST computed the same way (both sums of squared differences), so the ratio is meaningful? 4. For polynomial regression, am I treating x^2 (etc.) as just another ordinary feature, rather than assuming a different training process is needed? 5. If a straight-line fit has stubbornly high MSE, have I checked whether the data's SHAPE curves, before assuming more training or a different learning rate would help?